

Task 1. Identify Key Concepts

- a) List and briefly define three key concepts of modern frontend frameworks, such as component-based design, client-side rendering, and declarative programming.
- b) For each concept, provide a short example or scenario where it would be beneficial in a real-world application.

- 1) **Component Based Design:** Component based design creates the reusable UI components that encapsulate their own HTML, CSS and JavaScript. Thus, the UI is broken down into smaller independent pieces which are reusable. Each component manages its own structure (HTML/JSS), style (CSS), and logic (JavaScript).

Real-time Scenario for Component Based Design:

e-Commerce website

In any e-Commerce website, we see the list of products available for shopping. The information about the product name, image, description, price etc. is provided for each product. Instead of repeating the code for every product, a **ProductCard** component is created, and it is re-used for each product.

If there is any layout or functionality change to be done, updating the 'ProductCard' would reflect the changes for all the products instantly and automatically.

- Provides modularity and maintainability
- Allows code reusability across different pages
- Helps with faster development and easier testing

Component Based Design makes e-Commerce websites efficient to build, maintain, and scale. It enables fast development, customization, and consistent user experience across devices and pages.

- 2) **Client-side Rendering:** This is a method where initial load fetches the web application, using JavaScript. The Data to be displayed on the page is fetched separately via web services. Instead of loading the entire HTML for each page, JavaScript renders and generates UI programmatically, leading to reduced server load and latency. Thus, the browser downloads minimal HTML and then the JavaScript renders and updates the UI on the client side.

The advantages of Client-side Rendering are,

- It reduces the data usage on both the server and client, leading to lower cloud hosting expenses and reduced mobile data usage for users.

- The lower data transfer results in shorter waiting time, which enhances the user experience significantly.
- Some of the computational load is shifted from the server to the client, which helps in lowering the server cost.

The disadvantages of Client-side Rendering are,

- The initial load to the website requires longer waiting time, as the JavaScript powering the entire site must be downloaded.
- The client devices need to have JavaScript support to handle the JavaScript processing. (Mostly all the modern browsers support the execution of modern JavaScript syntaxes, and hence this is not an issue now-a-days).
- The client devices need to do processing; hence they should be relatively powerful devices.

Real-time Scenario for Client-side Rendering:

Analytical Dashboards:

Analytical Dashboard is a user interface that provides insights into data trends, patterns, and metrics through visual elements like charts, graphs, tables, filters etc.

When we change any filter e.g. dates or departments etc., the Charts and other statistics get updated dynamically without reloading the entire page.

- Each element like chart, card etc. is developed as a self-contained component.
- Data is fetched from the APIs and rendered directly in the browser.
- When user interacts with it, e.g. changing the filters etc., then the page is updated dynamically without full page reloading.

- 3) **Declarative Programming:** The declarative approach allows the developers to specify the intended behavior of the application, such as binding the UI with the Data. The developers describe what UI should look like for a given state. The framework then takes care of updating the UI when the data changes. Developers do not need to manually code the DOM changes They need to simply declare what the UI should look like based on the current state.

Real-time Scenario for Declarative Programming:

Online Food Delivery App:

When users order food using online food delivery application, he/ she adds food items to the cart. Then certain UI changes are expected to be made like,

- Set the number of cart items
- Selected items should appear in the list
- When one or more items are added into the list, the 'Place Order' button should be enabled.

The code tracks the state changes (e.g. using `useState`). When the state changes, the component is re-evaluated, and the new output is compared with the previous one. Only changed parts of the actual DOM are updated.

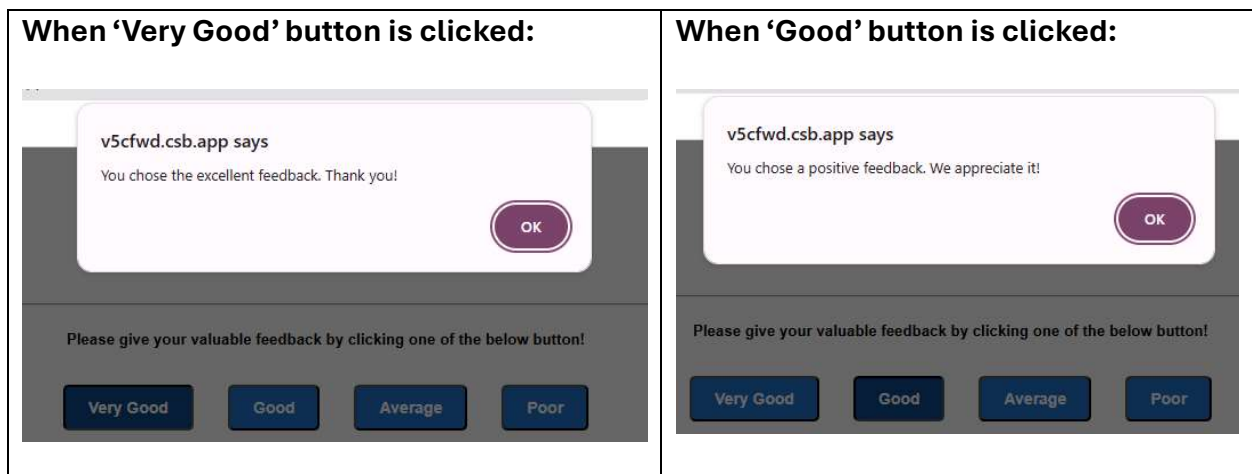
Task 2. Component Creation and Reusability

Please find the screenshots below. This example shows two components created and they are used multiple times in `App.js` file.

Button :

Button component is reused multiple times in `App.js` file.

Every button has different label, and every button click shows different alert messages, based on the value of input parameter (props) set in `App` file.



When 'Average' button is clicked:

v5cfwd.csb.app says

We understand that the huge improvement is required! Please believe in us.

OK

Please give your valuable feedback by clicking one of the below button!

Very Good

Good

Average

Poor

When 'Poor' button is clicked:

v5cfwd.csb.app says

We are sorry for the inconvenience! We will come back again with a better version!

OK

Please give your valuable feedback by clicking one of the below button!

Very Good

Good

Average

Poor

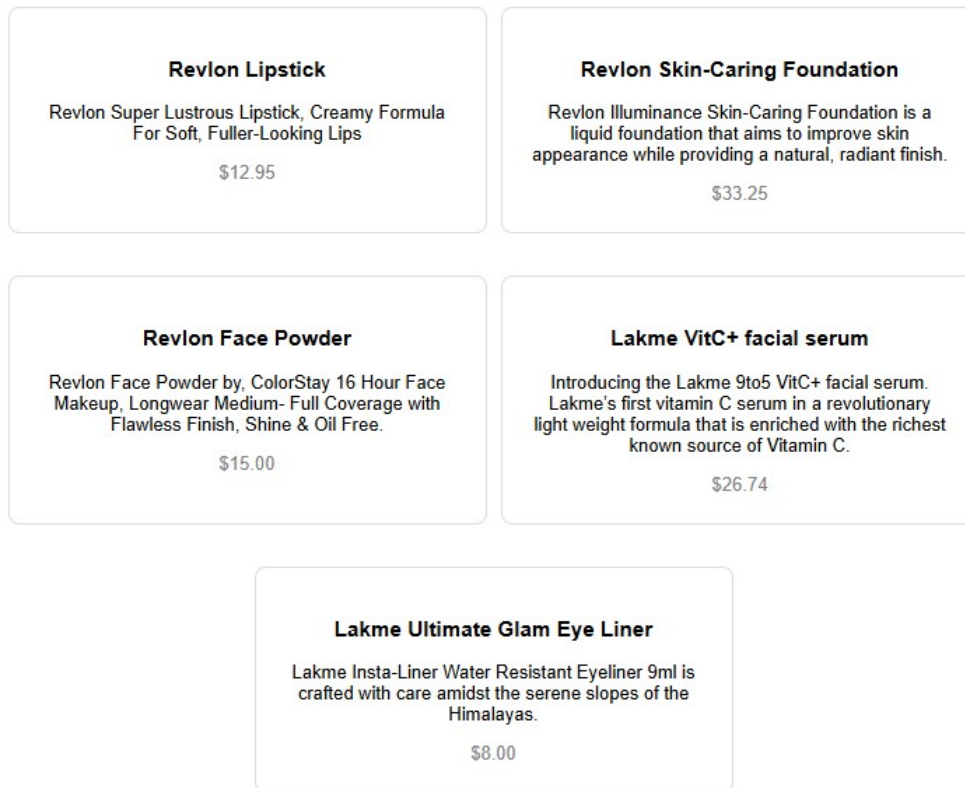
```
Button.js x
src > Components > Button.js > Button
1  import React from "react";
2  import "../styles.css";
3
4  /** Button Component */
5  function Button(props) {
6    const ButtonClicked = () => {
7      switch (props.ButtonLabel) {
8        case "Very Good":
9          alert("You chose the excellent feedback. Thank you!");
10         break;
11        case "Good":
12          alert("You chose a positive feedback. We appreciate it!");
13          break;
14        case "Average":
15          alert(
16            "We understand that the huge improvement is required! Please believe in us."
17          );
18          break;
19        case "Poor":
20          alert(
21            "We are sorry for the inconvenience! We will come back again with a better version!"
22          );
23          break;
24        default:
25          alert("Please choose your feedback!");
26          break;
27      }
28    };
29
30    return (
31      <div className="outerDiv">
32        <button className="ButtonComponent" onClick={ButtonClicked}>
33          {props.ButtonLabel}
34        </button>
35      </div>
36    );
37  }
38
39  export default Button;
40
```

ProductCard:

ProductCard component is reused multiple times in App.js file.

Every productCard displays different Product name, description and price based on input parameter (props) set in App file.

```
ProductCard.js X
src > Components > ProductCard.js > ...
1  import React from "react";
2  import "../styles.css";
3
4  /****** Card Component */
5  function ProductCard(props) {
6    return (
7      <div
8        style={{
9          border: "1px solid #ddd",
10         borderRadius: "8px",
11         padding: "20px",
12         marginBottom: "20px",
13         maxWidth: "300px",
14         fontSize: "13px",
15       }}
16      >
17        <h3>{props.productName}</h3>
18        <p>{props.productDesc}</p>
19        {
20          <div style={{ marginTop: "10px", color: "gray" }}>
21            ${props.productPrice}
22          </div>
23        }
24      </div>
25    );
26  }
27  export default ProductCard;
28
```



App.js

In App.js file, ProductCard component is used multiple times. In every usage ProductName, ProductDesc and ProductPrice is set. Hence same ProductCard component displays different information in different usage.

In App.js file, Button component is used multiple times. In every usage, different label is set for the button. Each button shows different alert messages when clicked. Thus, same Button component triggers different actions in every usage.

```

App.js x
src > App.js > ...
1 import "../styles.css";
2 import Button from "../Components/Button";
3 import ProductCard from "../Components/ProductCard";
4
5 export default function App() {
6   return (
7     <div className="App">
8       <h1>NUS Module 7</h1>
9       <h2>Task 2 : Component Creation and Reusability </h2>
10      <h3> Demo for reusable componets - Button and Card </h3>
11
12      <hr />
13      <h5>Below is our product list! Please take a look.</h5>
14      <div
15        style={{
16          display: "flex",
17          gap: "10px",
18          justifyContent: "center",
19          flexWrap: "wrap",
20        }}
21      >
22        <ProductCard
23          productName="Revlon Lipstick"
24          productDesc="Revlon Super Lustrous Lipstick, Creamy Formula For Soft, Fuller-Looking Lips"
25          productPrice="12.95"
26        />
27        <ProductCard
28          productName="Revlon Skin-Caring Foundation"
29          productDesc="Revlon Illuminance Skin-Caring Foundation is a liquid foundation that aims to improve skin appearance while providing a natural, radiant finish. "
30          productPrice="33.25"
31        />
32        <ProductCard
33          productName="Revlon Face Powder"
34          productDesc="Revlon Face Powder by, ColorStay 16 Hour Face Makeup, Longwear Medium- Full Coverage with Flawless Finish, Shine & Oil Free. "
35          productPrice="15.00"
36        />
37        <ProductCard
38          productName="Lakme VitC+ facial serum"
39          productDesc="Introducing the Lakme 9to5 VitC+ facial serum. Lakme's first vitamin C serum in a revolutionary light weight formula that is enriched with the richest known sou
40          productPrice="26.74"
41        />

```

```

App.js x
src > App.js > ...
5   export default function App() {
31     />
32     <ProductCard
33       productName="Revlon Face Powder"
34       productDesc="Revlon Face Powder by, ColorStay 16 Hour Face Makeup, Longwear Medium- Full Coverage with Flawless Finish, Shine & Oil Free. "
35       productPrice="15.00"
36     />
37     <ProductCard
38       productName="Lakme VitC+ facial serum"
39       productDesc="Introducing the Lakme 9to5 VitC+ facial serum. Lakme's first vitamin C serum in a revolutionary light weight formula that is enriched with the richest known sou
40       productPrice="26.74"
41     />
42     <ProductCard
43       productName="Lakme Ultimate Glam Eye Liner"
44       productDesc="Lakme Insta-Liner Water Resistant Eyeliner 9ml is crafted with care amidst the serene slopes of the Himalayas."
45       productPrice="8.00"
46     />
47   </div>
48   <hr />
49   <h5>
50     Please give your valuable feedback by clicking one of the below button!
51   </h5>
52   <div
53     style={{
54       display: "flex",
55       gap: "10px",
56       justifyContent: "center",
57       flexWrap: "wrap",
58     }}
59   >
60     <Button ButtonLabel="Very Good"></Button>
61     <Button ButtonLabel="Good"></Button>
62     <Button ButtonLabel="Average"></Button>
63     <Button ButtonLabel="Poor"></Button>
64   </div>
65 </div>
66 );
67 }
68

```

Below is what we can see when the page loads



NUS Module 7

Task 2 : Component Creation and Reusability

Demo for reusable componets - Button and Card

Below is our product list! Please take a look.

Revlon Lipstick

Revlon Super Lustrous Lipstick, Creamy Formula
For Soft, Fuller-Looking Lips

\$12.95

Revlon Skin-Caring Foundation

Revlon Illuminance Skin-Caring Foundation is a
liquid foundation that aims to improve skin
appearance while providing a natural, radiant finish.

\$33.25

Revlon Face Powder

Revlon Face Powder by, ColorStay 16 Hour Face
Makeup, Longwear Medium- Full Coverage with
Flawless Finish, Shine & Oil Free.

\$15.00

Lakme VitC+ facial serum

Introducing the Lakme 9to5 VitC+ facial serum.
Lakme's first vitamin C serum in a revolutionary
light weight formula that is enriched with the richest
known source of Vitamin C.

\$26.74

Lakme Ultimate Glam Eye Liner

Lakme Insta-Liner Water Resistant Eyeliner 9ml is
crafted with care amidst the serene slopes of the
Himalayas.

\$8.00

Please give your valuable feedback by clicking one of the below button!

Very Good

Good

Average

Poor

Task 3. CSR & SSR

a) Explain CSR & SSR

b) Difference between CSR and SSR with advantages

Client-Side Rendering (CSR): In CSR the browser loads a minimal HTML in initial load. The JavaScript then renders and updates the UI on the client side programmatically. All routing and rendering happen on the client side. E.g. React, Angular

Server-Side Rendering (SSR): In SSR, the server generates the full HTML for every request and sends it to the browser. Server is responsible for generating, rendering the HTML content and sending entire page to the client. Then JavaScript hydrates it for interactivity. The page is fully rendered on the server. If the pages need additional DOM manipulation, developers need to write additional JavaScript to manually update the DOM on specific events. E.g. PHP, ASP.Net

	Client-Side Rendering (CSR)	Server-Side Rendering (SSR)
1)	Rendering happens on the client side.	Rendering happens on the server side.
2)	The initial load to the website requires longer waiting time, as JavaScript powering the entire site must be downloaded.	The initial page load is faster.
3)	The subsequent navigations are faster as they are handled by client.	The subsequent navigations will be slower as they need to reload the page.
4)	Complexity: CSR is easier to set up and deploy.	Complexity: SSR are more complex. Also, any required DOM manipulation must be taken care by developers using additional JavaScript.
	Advantages of CSR: • It reduces the data usage on both the server and client, leading to lower cloud hosting expenses and reduced mobile data usage for users. • The lower data transfer results in shorter waiting time, which enhances the user experience significantly. • Some of the computational load is shifted from the server to the client,	Advantages of SSR: • Faster initial page loading. • This is better option for Search Engines. • Better performance on slow devices. • Contents are immediately available in the Dom and hence easily accessible.

	which helps in lowering the server cost. • It is a good option for interactive Apps. • It is easier to host. • Seamless client-side routing.	
	Dis-Advantages of CSR: • The initial load to the website requires longer waiting time, as the JavaScript powering the entire site must be downloaded. • User may see blank screen while JavaScript loads. • The client devices need to have JavaScript support to handle the JavaScript processing. (Mostly all the modern browsers support the execution of modern JavaScript syntaxes, and hence this is not an issue now-a-days). • The client devices need to do processing; hence they should be relatively powerful devices. • Search Engines may not index JavaScript rendered contents.	Dis-Advantages of SSR: • Difficult to manage large codebases • Tight coupling of backend and frontend technologies. • Backend and frontend developers need to work very closely which slows down the process. • Code is often written for specific pages which makes it difficult to re-use. It may lead to duplication and inconsistency. • Server generates entire page for every request causing the page to reload on every request. • This results in increased server load and latency. Causes higher data usage on both the server and the user. • Causes poor responsiveness and compromises the user experience. • Difficult to manage growing complexity.

Task 4. Single Page Application (SPA) Structure

a) Define SPA

b) List two advantages of SPA structure for modern web applications

Single Page Application (SPA) is a web application that loads a single HTML page, and dynamically updates the contents based on the user interactions. The navigation and content updates are handled by JavaScript (client-side). It does not require a full-page reload to change view or perform actions. The URL and contents are updated without reloading the page. Single Page Application fetches data from server. It is done asynchronously using technologies such as AJAX or Fetch API.

One example of Single Page Application is Gmail.

- In Gmail, the main page is loaded when we open the application.
- When we click on different links like Inbox, Sent, Drafts, Labels etc., the page is not fully refreshed.
- Instead, Javascript fetches new data and updates the interface which is very fast.
- It results in faster and smoother user experience.

Advantages of Single Page Application:

- It provides offline access to previously loaded content during network disruptions.
- After the initial load, page transitions are fast and instant.
- It enhances user experience through faster interactions by reducing server requests.
- The transition between the views is smoother. It provides more responsive and engaging interface for users.
- Asynchronous content loading
- Dynamic updates
- Client-side routing
- Components can be reused across pages.
- It is easier to separate frontend and backend and easier to work independently.

Dis-Advantages of Single Page Application:

- Single Page Application often have longer initial page load time, as the entire application needs to be loaded upfront.

- Search Engine may have difficulty indexing contents which are dynamically loaded.
- Older web browsers may not be compatible
- It is dependent on JavaScript.
- Complexity in state management